
External Services Permissions Layer for AI Agents

Role: Backend Engineering Intern (AI) **Stack:** Python (Django + Django ORM), Claude Agent SDK, SQLite (or any DB you prefer)

Background

Crator runs AI agents that connect to a variety of external tools and services, email, spreadsheets, messaging, web search APIs, browser automation, and so on. As we onboard more services, the way these integrations are wired up today is starting to feel risky.

Right now, when an agent is given access to a tool, it inherits everything that tool can do. If the agent is connected to a mailbox, it can read messages and send them and delete them with equal ease, and nobody but the agent itself decides which of those is appropriate at any given moment. There is no granular permission model, no audit trail of what the agent has actually been doing, and no way for a human to step in and review a sensitive action before it executes against the real world.

As a concrete example: imagine an agent that has access to some toolkit allowing it to read items, create items, update items, and delete items. The behavior we actually want is closer to read freely, update only after a human signs off, and never delete. The goal of this assignment is to build the layer that makes that kind of policy possible.

Scope

The deliverable is a permissions and audit layer that sits between agents and the external services they call. Concretely, the system you build should be capable of:

1. Maintaining a catalog of the external service toolkits available to agents and the individual actions each toolkit exposes, along with their input and output schemas.

2. Enforcing per-action permission rules at the moment an agent attempts to execute something, so that the agent cannot bypass the policy by going around the layer.
 3. Logging every attempt, successful, denied, or pending, to an immutable audit trail that survives even when the agent that made the call is later removed.
 4. Supporting a human-in-the-loop approval workflow for actions that have been classified as sensitive, including a way for the agent to wait for the decision and a way for a human to record approval or rejection.
 5. Authenticating agents via short-lived JWT tokens, so that every call into the layer is tied to a specific agent identity and a specific window of time.
-

What You Are Building

There are exactly two pieces to build, and the boundary between them matters.

The first piece is a **Django backend** that exposes the REST API described further down in this document. This backend is the permissions layer itself. It validates the JWT presented by each incoming request, looks up the relevant permission rule for the action being attempted, enforces that rule, writes a row to the audit log for every attempt regardless of outcome, and, when an action requires human review, holds the pending request in an approval queue until somebody resolves it. Use the Django ORM for persistence, and SQLite as the underlying database (any other database that Django supports is fine, but there is no reason to reach for one).

The second piece is a **Claude Agent SDK CLI app**, which acts as the agent in this story. The agent uses the Claude Agent SDK's standard built-in tools (`Read` , `Write` , `Edit` , and `Bash`), and on top of those you register exactly one custom LLM tool, `GenerateJWT` , which the agent calls to obtain a short-lived token from the Django backend. Once the agent has a JWT, every subsequent interaction with an external service happens through HTTP requests, typically `curl` invocations issued through the `Bash` tool, to the Django backend's REST endpoints.

The external services themselves (whichever toolkit you choose to wire up) are deliberately not exposed to the LLM as direct tool calls. The agent never gets a tool called `send_email` or `update_page` that it can invoke directly. Instead, it has to discover and call those actions through the Django backend, which is the only thing standing between the agent and the real world. This separation is the entire point of the layer: by funnelling every external action through one HTTP boundary, the permissions and audit logic gets a chance to run on every single call.

End-to-end workflow

The lifecycle of a single demo run looks roughly like this:

1. You stand up the Django app, with its catalog of toolkits and actions, its permission rules, its approval queue, its audit log, and the endpoints described below.
2. You stand up the Claude Agent SDK CLI, with `GenerateJWT` registered as a custom tool.
3. When the CLI runs, the agent's first move is to call `GenerateJWT` to obtain a short-lived token.
4. Carrying that JWT, the agent calls the backend to discover what toolkits and actions are available, and then attempts to call one of those actions.
5. Some actions return immediately, either with a successful result or with a `403` denial, based purely on the configured permission rules. You should demonstrate at least one action that the system flatly denies.
6. Other actions return a `202` indicating that human approval is required, along with a ticket ID. The agent polls the backend for the decision. Meanwhile, a human (you, in the demo) calls the `resolve/` endpoint directly with curl, Bruno, or Postman to approve or reject the ticket. Once resolved, the agent's poll returns the final outcome and the CLI prints it.

In a production version of this system, the `resolve/` endpoint would be hit by a real reviewer UI somewhere. For this assignment, calling it manually with an HTTP client is exactly the right level of fidelity.

Database Schema

Persistence is handled through the Django ORM. The exact field types and column names are your call, but the data described below has to be represented somewhere in the schema:

- **Agent.** Every agent in the system gets a row here, identified by a UUID primary key. Alongside the UUID, store a human-readable name (used purely for display and logging, e.g. "research-bot" or "sales-emailer"), an `is_active` flag for soft disabling, and a `created_at` timestamp. Every permission check, audit log row, and approval ticket in the system references an agent by its UUID; the name itself never participates in authorization.
- **Toolkit.** A toolkit represents one external service surface, Notion, GitHub, Slack, or whatever you've chosen. Each toolkit needs at minimum a slug, a display name, and a short description.
- **Action.** Each toolkit has multiple actions hanging off it. An action carries a slug, a display name, a description, an input schema and output schema (both stored as JSON), and a default permission, which is one of `always_allow`, `requires_approval`, or `always_deny`.

- **Permission override.** This table holds the sparse per-agent overrides of an action's default permission. The important constraint here is sparsity: do not pre-populate a row for every combination of agent and action. Only insert a row when a specific agent's effective permission for a specific action differs from that action's default. When resolving a permission at request time, the lookup is "is there an override row for this (agent, action)? If yes, use it; if no, fall back to the action's default."
- **Approval ticket.** When an agent attempts a `requires_approval` action, a row goes into this table. It stores a reference to the agent, a reference to the action, the params the agent submitted, the current status (one of `pending`, `approved`, `rejected`, or `expired`), the eventual result if the action ran, the rejection reason if any, the creation timestamp, and an expiry timestamp (a reasonable default is 24 hours from creation, after which the ticket auto-expires).
- **Audit log.** Every attempt to call an action, successful, denied, or pending, produces one row in this append-only table. Each row records the agent, the toolkit, the action, the params, the outcome, and a timestamp. This table has to be effectively immutable: there is no update path, and crucially, deleting an agent must not cascade-delete its log rows. The audit log is the record of what happened, and it has to survive operational events like agent cleanup.

For reference, the three permission values behave as follows:

Permission	Behavior
Always Allow	Action executes immediately when called.
Requires Approval	The call is held in the approval queue, and the agent polls for the decision.
Always Deny	The call is blocked immediately and returns a denial.

All of these models should be registered with the Django admin so that they're browsable from the admin UI without you needing to build anything custom on the frontend.

Agent Authentication (JWT)

Agents authenticate to the external services layer using short-lived JWT tokens, which keeps each session bounded in time and tied to a specific agent identity.

The flow looks like this. The `GenerateJWT` custom tool, which you register on the Claude Agent SDK side, calls a token issuance endpoint on the Django backend. The backend signs

and returns a JWT whose payload contains the agent's UUID (`agent_id`), the agent's display name (`agent_name` , included purely for logging and easier debugging), and an expiry timestamp (`exp`) somewhere in the 15 to 30 minute range. The agent then attaches that JWT as an `Authorization: Bearer <token>` header on every subsequent request it makes into the backend's external services API.

On the server side, every request that requires authentication runs through the same validation step: the signature has to verify against the backend's signing key, the token must not be expired, and the agent referenced by the token must still exist and be active. Any of those failing should return a clean `401` . Use `PyJWT` for the actual signing and verification work, there is no reason to roll your own.

API Endpoints

All endpoints below live on the Django backend. All require a JWT except `PATCH .../resolve/` (which is the human's endpoint and uses admin auth). The examples use a generic `<toolkit>` / `<action>` , pick any toolkit you like; ideally include at least one mutating action so the approval flow is meaningful.

List toolkits

```
GET /api/external-services/toolkits/
```

Response:

```
{
  "toolkits": [
    {"slug": "<toolkit_slug>", "name": "<Toolkit Name>", "description": "..."}
  ]
}
```

List actions in a toolkit

```
GET /api/external-services/toolkits/<toolkit>/actions/
```

Response:

```
{
  "toolkit": "<toolkit>",
  "actions": [
    {"slug": "read_item", "name": "Read", "description": "...", "permission": "..."},
    {"slug": "update_item", "name": "Update", "description": "...", "permission": "..."},
    {"slug": "delete_item", "name": "Delete", "description": "...", "permission": "..."}
  ]
}
```

Get action schema

```
GET /api/external-services/toolkits/<toolkit>/actions/<action>/schema/
```

Response:

```
{
  "toolkit": "<toolkit>",
  "action": "<action>",
  "input_schema": { "...": "JSON schema for params" },
  "output_schema": { "...": "JSON schema for result" }
}
```

Call an action

```
POST /api/external-services/toolkits/<toolkit>/actions/<action>/call/
```

```
{
  "params": { "...": "action-specific params" }
}
```

200 (allowed):

```
{"status": "executed", "result": { "...": "action-specific result" }}
```

202 (requires approval):

```
{"status": "pending_approval", "ticket_id": "<uuid>", "message": "..."}"
```

403 (denied):

```
{"status": "denied", "message": "This action is not permitted for this agent."}
```

Poll approval status (agent calls this while waiting)

```
GET /api/external-services/approvals/<ticket_id>/status/
```

```
{"status": "pending"}  
{"status": "approved", "result": { ... }}  
{"status": "rejected", "reason": "..."}  
{"status": "expired"}
```

Resolve approval (human's endpoint, no JWT, admin auth)

```
PATCH /api/external-services/approvals/<ticket_id>/resolve/
```

```
{"decision": "approved"}  
or  
{"decision": "rejected", "reason": "..."}
```

View/set per-agent permission overrides

```
GET /api/external-services/agents/<agent_id>/permissions/  
PUT /api/external-services/agents/<agent_id>/permissions/
```

Query audit log

```
GET /api/external-services/audit/?agent_id=<uuid>&toolkit=<slug>&outcome=denied
```

Tool Registration

The catalog (Toolkit + Action rows in your database) has to be populated somehow before the agent can call anything. For this assignment, that catalog should be filled from a real MCP server, pick whichever one you find interesting, as long as it exposes a meaningful mix of read and write actions. Notion, GitHub, Linear, Slack, and Google Drive all work well; the key requirement is that the server gives you enough surface area to set up at least one always-allow read action, at least one mutating action you can gate behind approval, and at least one destructive action you can deny outright. Picking an MCP server with only read actions defeats the purpose, because the approval and deny paths would never get exercised.

MCP (Model Context Protocol) is an open protocol for connecting AI agents to external tool servers. Every MCP server advertises the tools it offers, along with their input and output schemas, through a standard discovery interface. Your registration code should connect to the MCP server, enumerate the tools it exposes, and write each one into your catalog as a Toolkit (the server itself) and a set of Actions (the individual tools), copying over the JSON

schemas the server reports. You then assign each action a default permission, `always_allow`, `requires_approval`, or `always_deny`, either through a small management command, the Django admin, or a fixture you load at startup. Once the catalog is populated, the rest of the system doesn't care where the action originally came from: every call still goes through the same `POST .../call/` endpoint, hits the same permission check, generates the same audit log row, and (if needed) creates the same kind of approval ticket.

If wiring up a live MCP server turns out to be a hassle for any reason, it's acceptable to register the same toolkit through a small piece of Python code that mimics what the MCP discovery step would have produced, i.e. a function that inserts the toolkit row, inserts the action rows, and stores their schemas. The point of the registration layer is to demonstrate that the permissions pipeline is decoupled from the source of the tool definitions; how exactly those definitions get into the database is secondary.

Demo

The demo is a live walkthrough rather than a written report. You run the Claude Agent SDK CLI, give it a prompt that exercises the toolkit you've registered, and narrate what's happening as the agent interacts with the backend. The goal is to show all three permission outcomes (allow, requires-approval, deny) and the audit log working end to end in a single coherent session.

Before recording the demo, configure the toolkit you picked so that it has at least one read action set to **Always Allow**, at least one mutating action set to **Requires Approval**, and at least one destructive action set to **Always Deny**. This gives you all three branches of the permission logic to exercise on screen.

The flow to walk through:

1. The CLI starts up and the agent's first move is to call `GenerateJWT`, which returns a short-lived token.
2. Using that token, the agent uses `Bash` and `curl` to hit `GET /toolkits/` and `GET /toolkits/<toolkit>/actions/`, discovering what's available in the catalog.
3. The agent attempts the always-allow action. The backend returns `HTTP 200` with the result, and the CLI prints it.
4. The agent attempts the requires-approval action. The backend returns `HTTP 202` along with a `ticket_id`. The agent starts polling `GET .../status/`, where it will keep seeing `pending` until something changes.

5. You switch over to your HTTP client and hit `PATCH .../resolve/` directly with either an approval or a rejection. The agent's next poll picks up the resolved status, and the CLI prints whatever the outcome was.
6. The agent attempts the always-deny action. The backend returns `HTTP 403`, and the CLI prints that the action was denied.
7. Finally, open the Django admin (or call `GET /audit/`) and show that every single one of these attempts, the allowed one, the approved one, and the denied one, has produced an audit log entry capturing the agent, toolkit, action, params, outcome, and timestamp.

Evaluation

Criteria	
System design	Clean schema, separation of concerns
Security	Immutable logs, proper FK constraints, JWT validation
API design	RESTful, clear errors, correct status codes
Code quality	Readable, well-structured
Communication	Clear explanation of decisions in the video
Completeness	End-to-end demo works (allow + approval + deny + audit)

Not evaluated: frontend, deployment, performance optimization, complex concurrency handling.

Deliverables

1. **Video walkthrough**
2. Data model and design decisions
3. Live demo of the full flow (CLI → JWT → list toolkits → list actions → call action → allow / approval via `resolve/` endpoint / deny → audit log)
4. Code walkthrough of interesting parts (how `GenerateJWT` is registered as a custom Claude Agent SDK tool, how the permission check is wired into the `call/` endpoint, how MCP tools get registered into the catalog)
5. Tradeoffs and limitations

The video is the primary deliverable. Screen recording with voiceover is fine.

1. **GitHub repository**
 2. Django backend
 3. Claude Agent SDK CLI app (with `GenerateJWT` custom tool registered)
 4. README with setup instructions for both
-

Guidance

- Start with the data model.
- Sparse permissions, don't store a row for every (agent × action) pair, only overrides.
- The only custom LLM tool you need to register on the Claude Agent SDK side is `GenerateJWT`. `Read / Write / Edit / Bash` come built-in. Everything else (calling external services) happens via `Bash + curl`.
- MCP integration can use an existing SDK or be mocked, the point is that MCP tools go through the same pipeline as code-defined tools.
- Approval/rejection has no UI requirement, Django admin + the `PATCH .../resolve/` endpoint is enough.
- Keep the demo simple. One toolkit, ~4 actions, one agent, clean flow.
- `PyJWT` for token handling.
- A clean working system with 4 actions beats a complex half-working system with 20.
- Tests are not required. If you want to add some, that's fine.

Document assumptions in the README if anything is unclear.